NAME: S.KARTHIKEYAN **3122 22 5001 056**
CSE - B

UCS 2312 Data Structures Lab
Exercise 2: ListADT and its applications
Date of Exercise: 19.09.2023

Create an ADT for the linked list data structure with the following functions. list*ADT* will have the integer array and size.
[CO1, K3]

a. insert(header,data) – Insert data into the list using inserting at front
b. display(header) – Display the elements of the list
c. insertAtEnd(header,data) – Insert data at the end of the list
d. searchElt(header, key) – return the value if found, otherwise return -1
e. findMiddleElt(header) – find the middle element in the list
f. reverseList(header) – Reverse the list
g. detectLoop(header) – return the status of whether the loop is present or not
h. deleteElt(header,data) – Deletes the element data

Write a program in C to test the listADT for its operations with the following test cases.

Operation	**Expected Output**
length(header)	0
insert(header,2)	2
insert(header,4)	4, 2
insert(header,6)	6, 4, 2
insert(header,8)	8, 6, 4, 2
length(header)	4
insertLast(header,1)	8, 6, 4, 2, 1
insertLast(header,3)	8, 6, 4, 2, 1, 3
length(header)	6
findMiddleElt(header)	2 or 4
reverseList(header)	3, 1, 2, 4, 6, 8
searchElt(4)	4
searchElt(5)	-1
deleteElt(2)	8, 6, 4, 1, 3

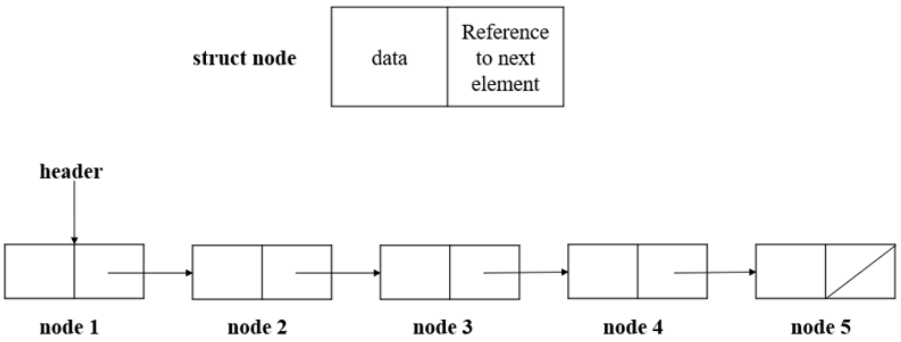
Best practices to be followed:

- Design before coding
- Usage of algorithm notation
- Use of multi-file C program
- Versioning of code

Write a program to subtract the given two polynomials
Example:
Poly1: $6x^7+8x^3+7x^2+9$,
Poly2: $7x^3+6$
Resultant Poly: $6x^7+15x^3+7x^2+15$

NAME: S.KARTHIKEYAN **3122 22 5001 056**
CSE - B

DATA STRUCTURE – LINKED LIST



ALGORITHMS:

1.FUNCTION NAME: insert
INPUT:
i)struct node *
ii)ele (int)
OUTPUT: None

```
1. struct node *temp;
2. temp=(struct node*)malloc(sizeof(struct node));
3. temp->data = ele;
4. struct node *ptr;
5. ptr = h->next;
6. if (ptr==NULL)
   {
   temp->next = h->next;
   h->next = temp;
    }
7. else
   {
   temp->next = h->next;
    h->next = temp;
   }
```

2.FUNCTION NAME: display
INPUT:
i)struct node *
OUTPUT: None

```
1. struct node *ptr;
2. ptr=h->next;
3. while(ptr!=NULL)
    {
    DISPLAY ptr->data
   ptr=ptr->next;
```

```
}
```

3.FUNCTION NAME: insertatend

INPUT:

i)struct node *
ii) ele (int)

OUTPUT: None

```
1. struct node *ptr,*prev;
2. ptr = h->next;
3. while (ptr != NULL)
   {
   prev = ptr;
   ptr = ptr->next;
   }
4. struct node *temp;
5. temp = (struct node *)malloc(sizeof(struct node ));
6. temp->data = ele;
7. temp->next = prev->next;
8. prev->next = temp;
```

4.FUNCTION NAME: searchElt

INPUT:

i)struct node *
ii)key(int)

OUTPUT: int

```
1. struct node *ptr;
2. ptr = h->next;
3. while (ptr!=NULL)
   {
   IF (ptr->data == key)
            return key;
   ptr=ptr->next;
   }
4. return -1;
```

5.FUNCTION NAME: findMiddleElt

INPUT:

i)struct node *

OUTPUT: None

```
1. struct node *ptr;
```

```
2.  ptr = h->next;
3.  int len = length(h);
4.  int k=0;
5.  if (len%2==0)
    {
        while (ptr !=NULL)
        {
            k=k+1;
            if (k==len/2 || k==len/2+1)
            {
                DISPLAY ptr->data
            }

            ptr=ptr->next;

        }
    }
6.  else
    {
        while (ptr !=NULL)
        {
            k=k+1;
            if (k==(len+1)/2)
            {
                DISPLAY ptr->data
            }

            ptr=ptr->next;

        }
    }
```

6.FUNCTION NAME: reverseList

INPUT:

i)struct node *

OUTPUT: None

```
1.  struct node *ptr,*temp;
2.  ptr = h->next;
3.  while (ptr->next!=NULL)
    {
        temp=h->next;
        h->next=ptr->next;
        ptr->next = (h->next)->next;
        (h->next)->next = temp;
    }
```

NAME: S.KARTHIKEYAN **3122 22 5001 056**
CSE - B

7.FUNCTION NAME: detectLoop
INPUT:
i)struct node *

OUTPUT: int

```
1. struct node *ptr;
2. ptr=h->next;
3. struct node *arr[100];
4. int i=0;
5. while(ptr!=NULL)
   {
        if (i==0)
        {arr[i]=ptr;
        i++;
        ptr=ptr->next;
   }
6. else
   {
        int count = 0;
        for (int j=0;j<i;j++)
        {
             if (ptr==arr[j])
             count+=1;

             if (count>0)
             return 1;
        }
         i++;
        ptr=ptr->next;
   }

   }
7. return 0;
```

8.FUNCTION NAME: deleteElt
INPUT:
i)struct node*
ii)ele (int)
OUTPUT: None

```
1. struct node *ptr;
2. struct node *prev;
3. int res = search(h,ele);
4. if (res != -1)
```

```
{ ptr=h->next;
prev=h->next;
int k =0;
while (ptr!=NULL)
{
if (ptr->data == ele)
{
        if (k==0)
        {
                h->next=ptr->next;
                break;
        }
        else
        {
                prev->next=ptr->next;
                break;
        }
}
else
{
        k++;
        prev=ptr;
        ptr=ptr->next;
}
}
}
```

5.
```
else
{
printf("Element doesnt exist!");
}
```

CODE:
“listadt.h”

```
#include <stdio.h>
#include <stdlib.h>
struct node
{
   int data;
   struct node *next;
};

void insertfront(struct node *h,int ele)
{
   struct node *temp;
   temp=(struct node*)malloc(sizeof(struct node));
   temp->data = ele;
```

```
   struct node *ptr;
   ptr = h->next;
   if (ptr==NULL)
   {
      temp->next = h->next;
      h->next = temp;
   }
   else
   {
      temp->next = h->next;
      h->next = temp;
   }
}

void display(struct node *h)
{
   struct node *ptr;
   ptr=h->next;
   while(ptr!=NULL)
   {
      printf("%d ",ptr->data);
      ptr=ptr->next;
   }
}

void insertAtEnd(struct node *h,int ele)
{
  struct node *ptr,*prev;
  ptr = h->next;
  while (ptr != NULL)
    {
      prev = ptr;
      ptr = ptr->next;
    }
  struct node *temp;
  temp = (struct node *)malloc(sizeof(struct node ));
  temp->data = ele;
  temp->next = prev->next;
  prev->next = temp;
}

int search(struct node *h,int key)
{
  struct node *ptr;
  ptr = h->next;
  while (ptr!=NULL)
    {
```

```
    if (ptr->data == key)
      return key;
    ptr=ptr->next;
   }
  return -1;

}

void findMiddleElt(struct node *h)
{
  struct node *ptr;
  ptr = h->next;
  int len = length(h);
  int k=0;
  if (len%2==0)
  {
    while (ptr !=NULL)
     {
      k=k+1;
      if (k==len/2 || k==len/2+1)
      {
        printf("%d ",ptr->data);
      }

      ptr=ptr->next;

     }
  }
  else
  {
    while (ptr !=NULL)
     {
      k=k+1;
      if (k==(len+1)/2)
      {
        printf("%d ",ptr->data);
      }

      ptr=ptr->next;

     }
  }
}
void reverseList(struct node *h)
{
  struct node *ptr,*temp;
  ptr = h->next;
```

```
  while (ptr->next!=NULL)
   {
     temp=h->next;
     h->next=ptr->next;
     ptr->next = (h->next)->next;
     (h->next)->next = temp;
   }

}

void deleteElt(struct node *h,int ele)
{ struct node *ptr;
  struct node *prev;
  int res = search(h,ele);
  if (res != -1)
  { ptr=h->next;
  prev=h->next;
   int k =0;
  while (ptr!=NULL)
   {
     if (ptr->data == ele)
     {
       if (k==0)
       {
         h->next=ptr->next;
         break;
       }
       else
       {
      prev->next=ptr->next;
      break;
       }
     }
     else
     {
       k++;
       prev=ptr;
       ptr=ptr->next;
     }
    }
  }
  else
  {
    printf("Element doesnt exist!");
  }
}
```

NAME: S.KARTHIKEYAN **3122 22 5001 056**
CSE - B

```
void tocorrupt(struct node *h,int k)
{
   struct node *ptr,*end,*prev;
   int k1=0;
   ptr=h->next;
   while(ptr!=NULL)
   {
      k1++;
      if (k1==k)
         {
            end=ptr;
         }
      else
        {
         prev=ptr;
        ptr=ptr->next;
        }
   }
   prev->next = end;
}

int detectLoop(struct node *h)
{
   struct node *ptr;
   ptr=h->next;
   struct node *arr[100];
   int i=0;
   while(ptr!=NULL)
   {
      if (i==0)
         {arr[i]=ptr;
         i++;
         ptr=ptr->next;
         }
      else
      {
         int count = 0;
         for (int j=0;j<i;j++)
         {
            if (ptr==arr[j])
               count+=1;

            if (count>0)
               return 1;
         }
         i++;
         ptr=ptr->next;
```

```
      }

   }
   return 0;
}
```

“main.c”

```
#include <stdio.h>
#include "listadt.h"
#include <stdlib.h>
void main()
{
 struct node *header;
 header = (struct node *)malloc(sizeof(struct node));
 header->next = NULL;
 printf("----INSERTION AT FRONT ----\n");
 printf("Enter how many elements you want to enter :");
 int n;
 scanf("%d",&n);
 for (int i=0;i<n;i++)
  {
    int a;
    printf("enter the element :");
    scanf("%d",&a);
    insertfront(header,a);
  }
printf("\n");
 printf("-----DISPLAY----\n");
 display(header);
 printf("\n");
  printf("----INSERTION AT END ----\n");
 printf("Enter how many elements you want to enter :");
 int n1;
 scanf("%d",&n1);
 for (int i=0;i<n1;i++)
  {
    int a;
    printf("enter the element :");
    scanf("%d",&a);
    insertAtEnd(header,a);
  }
printf("\n");
 printf("-----DISPLAY----\n");
 display(header);
 printf("\n");
 printf("\n\n-----SEARCHING FOR AN ELEMENT -----\n ");
```

```
 int ch;
 printf("Enter the element to be searched :\n");
 scanf("%d",&ch);
 int res = search(header,ch);
 if (res!=-1)
    printf("%d \n",res);
 else
  printf("%d \n",res);
 printf("\n");
 display(header);
 printf("\n----FINDING MIDDLE ELEMENT ----\n");
 findMiddleElt(header);
 printf("\n");
display(header);
 display(header);
 printf("\n\n---REVERSING A LINKED LIST-----\n");
 reverseList(header);
 printf("\n");
 display(header);
 printf("\n----DELETION OF AN ELEMENT ----\n");
 int elt;
 printf("Enter the deleted element :");
 scanf("%d",&elt);
 deleteElt(header,elt);
 printf("\n");
 display(header);
 tocorrupt(header,2);
 printf("\nLIST GETTING CORRUPTED....\n");
 printf("--CORRUPTED LIST ?----\n");
 printf("%d ",detectLoop(header));
 }
```

NAME: S.KARTHIKEYAN **3122 22 5001 056**

CSE - B

OUTPUT:

```
C:\Users\User\OneDrive\SEM3\DATA STRUCTURES\LAB\LINKED LIST>f1.exe
----INSERTION AT FRONT ----
Enter how many elements you want to enter :5
enter the element :5
enter the element :4
enter the element :3
enter the element :2
enter the element :1

-----DISPLAY----
1 2 3 4 5
----INSERTION AT END ----
Enter how many elements you want to enter :2
enter the element :56
enter the element :54

-----DISPLAY----
1 2 3 4 5 56 54


-----SEARCHING FOR AN ELEMENT -----
 Enter the element to be searched :
3
3

1 2 3 4 5 56 54
----FINDING MIDDLE ELEMENT ----
4
1 2 3 4 5 56 54

---REVERSING A LINKED LIST-----

54 56 5 4 3 2 1
```

```
54 56 5 4 3 2 1
----DELETION OF AN ELEMENT ----
Enter the deleted element :56

54 5 4 3 2 1
LIST GETTING CORRUPTED....
--CORRUPTED LIST ?----
1
```

NAME: S.KARTHIKEYAN **3122 22 5001 056**
CSE - B

QUESTION 2: Write a program to subtract the given two polynomials
Example:
Poly1: $6x^7+8x^3+7x^2+9$,
Poly2: $7x^3+6$
Resultant Poly: $6x^7+15x^3+7x^2+15$

CODE:

Main.c

```
#include <stdio.h>

#include <stdlib.h>

#include "polyadt.h"


void main()

{

        struct poly *header;

        header = (struct poly*)malloc(sizeof(struct poly));

 header->next = NULL;

        struct poly *header1;

        header1 = (struct poly*)malloc(sizeof(struct poly));

 header1->next = NULL;

         printf("enter the number of terms of P1:");

        int n1;

        scanf("%d",&n1);

         printf("enter the number of terms of P2:");

        int n2;

        scanf("%d",&n2);

        printf("----POLYNOMIAL 1 ----\n");

 for (int i=0;i<n1;i++)

  {

    printf("TERM %d\n",(i+1));

    printf("enter the coeff :");
```

```
    int cf;
    scanf("%d",&cf);
    printf("enter the exponent :");
    int ex;
    scanf("%d",&ex);
    insert(header,cf,ex);
  }

 printf("\n----POLYNOMIAL 2 ----\n");
 for (int i=0;i<n2;i++)
  {
    printf("TERM %d\n",(i+1));
    printf("enter the coeff :");
    int cf;
    scanf("%d",&cf);
    printf("enter the exponent :");
    int ex;
    scanf("%d",&ex);
    insert(header1,cf,ex);
  }

 printf("\n---DISPLAYING---\n");
 printf("\n----POLYNOMIAL 1 ----\n");
 display(header);
 printf("\n----POLYNOMIAL 2 ----\n");
 display(header1);
 printf("\n\n---ADDITION----\n");
 add(header,header1);
```

```
}
```

Polyadt.h

```
#include <stdio.h>
#include <stdlib.h>

struct poly

{          int coeff;

           int exp;

           struct poly *next;

};

void insert(struct poly *h,int c,int e)

{struct poly *ptr;

 ptr = h->next;

 if (ptr == NULL)

 {

   struct poly *temp;

 temp=(struct poly*)malloc(sizeof(struct poly));

 temp->coeff = c;

 temp->exp = e;

  temp->next = h->next;

  h->next = temp;

 }

 else

 { struct poly *temp;

  temp=(struct poly*)malloc(sizeof(struct poly));

  temp->coeff = c;

  temp->exp = e;

  struct poly *prev;

  prev = h->next;

  while (ptr!=NULL)

    {
```

```
      prev = ptr;
      ptr = ptr->next;
     }
   temp->next = prev->next;
   prev->next = temp;
 }}
void display(struct poly *h)
{
          struct poly *ptr;
          ptr = h->next;
          while (ptr!=NULL)
                    {
                              printf("%d x^%d + ",ptr->coeff,ptr->exp);
                              ptr=ptr->next;
                    }
}
void add(struct poly *h1,struct poly *h2)
{
 struct poly *p1,*p2;
 p1 = h1->next;
 p2 = h2->next;
 struct poly *sum;
 sum=(struct poly*)malloc(sizeof(struct poly));
 sum->next = NULL;
 while ((p1!=NULL) && (p2!=NULL))
  {
    if (p1->exp == p2->exp)
    {
      insert(sum,p1->coeff+p2->coeff,p1->exp);
```

```
      p1=p1->next;
      p2=p2->next;
    }
    else if (p1->exp > p2->exp)
    {
      insert(sum,p1->coeff,p1->exp);
      p1=p1->next;
    }
    else if(p1->exp < p2->exp)
    {
      insert(sum,p2->exp,p2->exp);
      p2=p2->next;
    }
  }

 while (p1!=NULL)
  {
    insert(sum,p1->coeff,p1->exp);
    p1=p1->next;
  }
 while (p2!=NULL)
  {
    insert(sum,p2->coeff,p2->exp);
    p2=p2->next;
  }
 display(sum);
}
```

NAME: S.KARTHIKEYAN **3122 22 5001 056**
CSE - B

OUTPUT:

```
enter the number of terms of P1:3
enter the number of terms of P2:3
----POLYNOMIAL 1 ----
TERM 1
enter the coeff :3
enter the exponent :3
TERM 2
enter the coeff :2
enter the exponent :2
TERM 3
enter the coeff :1
enter the exponent :1

----POLYNOMIAL 2 ----
TERM 1
enter the coeff :3
enter the exponent :3
TERM 2
enter the coeff :2
enter the exponent :2
TERM 3
enter the coeff :1
enter the exponent :0
```

```
---DISPLAYING---

----POLYNOMIAL 1 ----
3 x^3 + 2 x^2 + 1 x^1 +
----POLYNOMIAL 2 ----
3 x^3 + 2 x^2 + 1 x^0 +

---ADDITION----
6 x^3 + 4 x^2 + 1 x^1 + 1 x^0 +
```

NAME: S.KARTHIKEYAN **3122 22 5001 056**
CSE - B

NAME: S.KARTHIKEYAN **3122 22 5001 056**
CSE - B